1. What is Angular and how does it support SPA?

Angular is a TypeScript-based framework for building Single Page Applications (SPAs). It loads components dynamically without refreshing the page.

2. What is Angular CLI and why is it useful?

Angular CLI is a command-line tool that helps scaffold, build, test, and deploy Angular apps. It simplifies project setup and automation.

3. What is a component in Angular?

A component is the basic building block of Angular UI. It includes a TypeScript class, HTML template, and CSS styles.

4. How do you implement routing in Angular 18?

We use `RouterModule` with `routes` array and define paths using `path` and `component`. Angular 18 supports standalone components in routing.

5. What is interpolation in Angular?

Interpolation is one-way data binding using `{{ }}` syntax. It binds component properties to the template for display.

6. What is property binding in Angular?

Property binding uses square brackets `[ ]` to bind component data to HTML element properties like `[src]` or `[disabled]`.

7. What is two-way data binding?

Two-way binding uses `[(ngModel)]` to sync data between the component and the template. Changes reflect both ways instantly.

8. What is a structural directive?

Structural directives like `*ngIf` and `*ngFor` change the DOM layout by adding or removing elements dynamically.

9. What is an attribute directive?

Attribute directives like `ngClass` or custom ones modify the appearance or behavior of elements without changing structure.

10. What are pipes in Angular?

Pipes transform data in templates. Built-in pipes include `date`, `uppercase`, and `currency`. We can also create custom pipes.

11. What is control flow in Angular 17+?

Angular 17+ introduces control flow blocks like `@if`, `@for`, and `@switch`, improving readability and performance over traditional directives.

12. What is a signal in Angular?

Signals are reactive primitives introduced in Angular 17+. They track state changes and trigger updates efficiently in the UI.

13. What is a service in Angular?

A service is a class that holds business logic or shared data. It's injected into components to promote reusability and separation of concerns.

14. What is dependency injection (DI) in Angular?

DI is a design pattern where Angular provides required services to components automatically. It improves modularity and testability.

15. What are lifecycle hooks in Angular?

Lifecycle hooks like `ngOnInit`, `ngOnDestroy`, and `ngAfterViewInit` let us tap into different stages of a component's existence.

16. How do services interact with lifecycle hooks?

Services don't have lifecycle hooks directly, but they can be used inside components that implement hooks to manage data or subscriptions.

17. What is deep routing in Angular?

Deep routing allows nested routes using child route configurations. It helps build multi-level navigation structures in complex apps.

18. What are guards in Angular?

Guards like `CanActivate` and `CanDeactivate` control access to routes. They're used for authentication, authorization, and navigation control.

19. How does AuthService work with JSON?

AuthService typically sends login credentials via HTTP POST and receives a JSON response with tokens or user data for authentication.

20. What is an interceptor in Angular?

Interceptors are used to modify HTTP requests or responses. Common use cases include adding auth tokens or handling errors globally.

21. How do you send a token in the header?

We use an HTTP interceptor to attach the token to the `Authorization` header in every outgoing request using `HttpHeaders`.

22. What is the role of NgModules in Angular?

NgModules organize code into cohesive blocks. They help manage declarations, imports, providers, and bootstrap logic for modular architecture.

23. How does Angular's change detection work?

Angular uses zone.js to detect changes and trigger view updates. It compares component state with the template and re-renders only affected parts.

24. What is OnPush change detection strategy?

OnPush tells Angular to check for changes only when input properties change or events occur. It improves performance in large apps.

25. What is the difference between ViewChild and ContentChild?

ViewChild accesses elements inside the component's template. ContentChild accesses projected content passed via `<ng-content>`.

26. How does Angular handle lazy loading?

Lazy loading splits the app into feature modules loaded on demand via the router. It improves initial load time and performance.

27. What is standalone component architecture?

Standalone components in Angular 15+ don't require NgModules. They simplify structure and improve tree-shaking and code clarity.

28. What is the role of Signals in Angular 17+?

Signals are reactive primitives that track state changes. They replace zone.js and offer fine-grained reactivity with better performance.

29. How do you create a custom directive?

We use `@Directive()` decorator and implement logic inside the class. It can modify element behavior or appearance dynamically.

30. What is the difference between `ngIf` and `@if`?

`ngIf` is the traditional structural directive. `@if` is part of Angular 17+ control flow syntax, offering better performance and readability.

31. How does Angular handle form validation?

Angular supports template-driven and reactive forms. Validators like `required`, `minLength`, and custom ones ensure input correctness.

32. What is a resolver in Angular routing?

Resolvers fetch data before activating a route. They ensure components have required data upfront, improving UX and reducing flicker.

33. How do you implement guards for role-based access?

Use `CanActivate` with logic to check user roles from AuthService. Redirect unauthorized users to login or error pages.

34. What is the role of `HttpClient` in Angular?

`HttpClient` is Angular's built-in service for making HTTP requests. It supports observables, interceptors, and typed responses.

35. How do interceptors handle global errors?

Interceptors catch HTTP errors and handle them centrally. We can log errors, show alerts, or redirect users based on status codes.

36. What is the difference between `ngOnInit` and constructor?

Constructor initializes class members. `ngOnInit` is a lifecycle hook used for component setup after Angular initializes inputs.

37. How do you share data between components?

Use Input/Output decorators for parent-child communication. For unrelated components, use a shared service with observables or signals.

38. What is the role of `ngZone`?

`NgZone` helps Angular detect async operations outside its context. It can be used to manually trigger change detection when needed.

39. How does Angular handle SSR (Server-Side Rendering)?

Angular Universal enables SSR by rendering HTML on the server. It improves SEO and initial load performance for public-facing apps.

40. What is hydration in Angular SSR?

Hydration attaches Angular logic to server-rendered HTML. It enables interactivity without re-rendering the entire DOM on the client.